Advanced Security for NextG Mobile Networks: A Hybrid Fuzzing Approach

Jingda Yang, *Graduate Student Member, IEEE*, Paul Ratazzi, *Senior Member, IEEE*, and Ying Wang, *Member, IEEE*

***Abstract*—This paper presents HyFuzz, a hybrid intelligent fuzz testing platform designed to enhance the security validation of next generation (NextG) mobile networks. HyFuzz integrates symbolic formal analysis with adaptive fuzzing to enable the discovery of vulnerabilities that emerge from subtle state inconsistencies and session level command manipulations. Specifically, HyFuzz demonstrates support for multi step intra session fuzzing, where carefully crafted command sequences cause persistent state desynchronization between User Equipment (UE) and the network. Complementing this, HyFuzz employs formal guided deep fuzzing, directing fuzzing efforts to high risk protocol states identified by symbolic analysis. Through a dual mode architecture supporting both virtual (ZMQ) and over the air (OTA) fuzzing, HyFuzz provides an extensible testbed for low level and behavioral vulnerability discovery. Experimental results across 1,281 test cases reveal 1,105 failure instances, including stealthy failures that manifest only under extended interaction contexts. Our findings suggest HyFuzz provides a foundational capability toward more realistic and semantically rich vulnerability detection in modern mobile infrastructure.**

***Index Terms*—Non-intrusive platform, fuzz testing, virtualization, over-the-air, vulnerability.**

I. Introduction

THE increasing complexity and interconnectedness of 5G and NextG networks demand robust security measures to mitigate emerging threats and vulnerabilities. In addition, the evolution of 5G mobile networks towards a server-based architecture (SBA) comes with the emergence of numerous testing challenges [1]. Although softwarization and network function virtualization (NFV) adds flexibility and enables orchestration, it also requires the testing to evaluate not only the traditional communications domain specifications, but also software engineering quality. In addition to security, the concerned quality attributes include performance, modifiability, interoperability, reliability, etc. [2] To address these multifaceted challenges, fuzzing emerges as a vital tool in the testing arsenal. Fuzzing involves sending random or unexpected data inputs to a system to uncover vulnerabilities, unexpected behaviors, or system crashes. Within the context of the sophisticated 5G and NextG networks, especially with the adoption of server-based architectures and NFV, fuzzing plays a pivotal role in uncovering domain and software vulnerabilities and identifying potential weaknesses in general software stacks and network infrastructures. The assurance provided in the vigorous testing is crucial to securing 5G infrastructure protocols and stack.

In addition, experimental work in the context of 5G and NextG has gained significant attention over the past few years, shifting from simulation-driven research used in previous mobile network generations to system implementation and prototyping [3]. This trend parallels efforts in other domains to adopt systematic, AI-augmented pipelines for large-scale knowledge exploration [4]. This change stems from several factors, including the widespread adoption of programmable software defined radio (SDR), NFV, and subsequent open-source softwarization of mobile network functions through various projects such as OpenAirInterface (OAI) [5] and srsRAN [6]. This change enables intensive testing emulating real-world environments, and assessment of the System Under Test (SUT) when interacting with external platforms of critical infrastructures where it is essential to assess risks and performance impacts associated with potential vulnerabilities. Existing research in this vain includes unmanned aerial system (UAV) applications [7], healthcare and medical applications [8], [9], [10] and other applications.

The 3GPP test specifications list the requirements for both functional and non-functional testing in 5G network products. TS 33.117, Catalogue of General Security Assurance Requirements [11] requires network products to provide externally reachable services in spite of unexpected inputs. While formal verification [12] and domain knowledge analysis can provide mathematical proofs of security, it is worth noting that many vulnerabilities can only be effectively detected through fuzz testing in simulation platforms. Previous work to address 5G and NextG test specifications includes intrusion detection systems [13], [14], fuzz testing core services and Radio Access Networks (RANs) [1], [15], [16], [17], and open source tools such as Tcpreplay [18]. Though the efforts from these studies made important contributions in detecting existing vulnerabilities in 5G systems, it is important to note that they are limited to one-shot fuzzing intrusion. Moreover, they constrain vulnerability detection to be triggered by single-bit mutation or single-command misplacement while neglecting more complicated multi-step attack

Received 8 January 2025; revised 16 September 2025; accepted 22 September 2025. Date of publication 25 September 2025; date of current version 4 February 2026. This work was supported in part by Defense Advanced Research Projects Agency (DARPA) under Grant D22AP00144 and in part by Air Force Research Laboratory (AFRL) Summer Faculty Fellowship Program (SFFP) under Contract FA8750-20-3-1003. Recommended for acceptance by A. A. Nayak. *(Corresponding author: Ying Wang.)*

Jingda Yang and Ying Wang are with the Department of Systems and Enterprises, Stevens Institute of Technology, Hoboken, NJ 07030 USA (e-mail: jyang76@stevens.edu; ywang6@stevens.edu).

Paul Ratazzi is with the Information Warfare Division of the Air Force Research Laboratory, Rome, NY 13441 USA (e-mail: edward.ratazzi@afrl.af.mil).

Digital Object Identifier 10.1109/TMC.2025.3614127

strategies. Multi-step attack strategies, such as those associated with advanced persistent threats (APTs), are challenging to discover because they employ a series of covert actions, often using legitimate tools, and adapt to defenses in real-time. Their stealthy, diverse, and adaptable nature makes them harder to be identified than single-step attacks. However, discovering and understanding their tactics is key to developing truly effective defense mechanisms.

This paper strives to bridge the gap between emerging attack strategies and state of the art defense methods for both license-based 5G systems and open-source solutions by establishing a benchmark for multi-step fuzzing and performance assessment. Our data-driven approach is realized in our NextG hybrid testing platform for fuzz testing and performance evaluation, HyFuzz. HyFuzz not only provides 5G and NextG vulnerability discovery in a dynamic environment where protocol and software stack releases are frequent and continuous, but it also accelerates interdisciplinary research related to 5G assurance by revealing unintended emergent behaviors. HyFuzz interoperates with multiple radio platforms, including OAI, srsRAN, and can be operated in both over-the-air (OTA) and ZeroMQ (ZMQ) [19] mode. The ZMQ mode is a software-based method that allows signals to be passed through software interfaces, like Internet Protocol (IP) sockets. HyFuzz's integrated approach leverages the strengths of both command-level and bit-level fuzzing to target both confidentiality and integrity types of attacks, resulting in a comprehensive assessment of the security of 5G networks.

Three primary types of fuzzing – bit-level, command-level, and a special type of command-level called permutation-based fuzzing – are crucial for ensuring the security of communication protocols. Fuzzing systematically manipulates commands or messages to uncover vulnerabilities in protocol logic, command parsing, and processing flow [20]. By altering command parameters or targeting individual bits within the data payload, fuzzing can expose flaws that lead to unauthorized access, data breaches, or system malfunctions [21], [22]. Bit-level fuzzing approaches such as Berserker [15] and T-Fuzz [23] have identified critical vulnerabilities. Similarly, techniques like LTEInspector [24], LTEFuzz [25], 5GReasoner [26], Listen-and-Learn [27], and BaseSAFE [28] have demonstrated the efficacy of command-level fuzzing. Integrating command-level and bit-level fuzzing provides a holistic security assessment, ensuring the robustness and resilience of 5G systems against potential threats [16].

Continuous monitoring of digital twins provides real-time feedback on system behavior under various fuzzing inputs, identifying subtle issues missed in traditional testing. Fuller et al. [29] highlight the reliability benefits of continuous feedback loops in digital twins. Additionally, digital twins can simulate complex scenarios that are difficult to replicate physically, exposing software to a wider range of faults and edge cases. Glaessgen and Stargel [30] emphasize their utility in comprehensive testing and validation processes.

More broadly, the United States (US) Department of Defense (DoD) has begun to prioritize the use of digital engineering methodologies, technologies, and practices throughout the life cycle to support research, engineering, and management activities. DoD recognizes the role of models as the *"...basis for defining, evaluating, comparing, optimizing alternatives, [and enabling] concurrent engineering..."*[31] Digital twins based on authoritative sources of information, and combined with data collected from the corresponding physical system, sensor information, and inputs that mirror system operation are key to realizing this strategy [32].

Combining fuzz testing with modeling provides an opportunity to continuously refine and enhance digital twin models, making them more accurate and representative of corresponding real-world systems. Liu et al. [33] describe methods for using test data to enhance model accuracy and predictive capabilities. Fuzz testing results highlight discrepancies between the digital twin and the actual system, ensuring accurate reflections of real-world behavior. An iterative feedback loop from fuzz testing results continuously enhances both the digital twin and the software, ensuring they remain aligned with evolving operational realities and emerging threats. Grieves and Vickers [34] stress the importance of this iterative process for ongoing improvement.

In addressing the challenges of emulating close-to-real-life attacks and systematically detecting vulnerabilities in 5G and future communication protocols, we present HyFuzz and its digital twin platform in this paper. The contributions of this paper can be summarized as follows:

- We present HyFuzz, a hybrid fuzz testing platform that integrates both virtual relay based simulation and over the air (OTA) fuzzing environments to evaluate 5G protocol implementations under realistic and controllable conditions.
- We propose a multi step intra session fuzzing methodology that enables the injection and manipulation of sequential protocol messages within a single connection session. This reveals protocol state desynchronization effects and vulnerability conditions that single frame fuzzing cannot expose.
- We demonstrate that given semantically high risk protocol states derived through formal or specification based analysis, HyFuzz can execute structured fuzzing campaigns that effectively probe those states through both command level and bit level mutation.
- We introduce a modular digital twin architecture that enables consistent execution traces across relay modes, integration with external seed sources, and modeling of fuzz induced state transitions using Markov chains.
- We evaluate HyFuzz across 1,281 fuzzing cases, identifying 1,105 failure instances across 16 categories, including session aware failures that depend on prior message history and message order, which are missed by one shot fuzzers.

Fig. 1 provides an overview of the HyFuzz framework, illustrating how the Digital Twin and OTA platforms are integrated. This figure also serves as a roadmap for the sections that follow. The rest of the paper is organized as follows. Section II covers background information and related work on fuzzing strategies. Section III details the system architecture and digital twin platform. Section IV describes the HyFuzz hybrid system model, including ZMQ and OTA modes. Section V discusses multi-mode fuzz testing methodologies. Section VI presents the experimental results, and Section VII concludes the paper with a summary and future work directions.

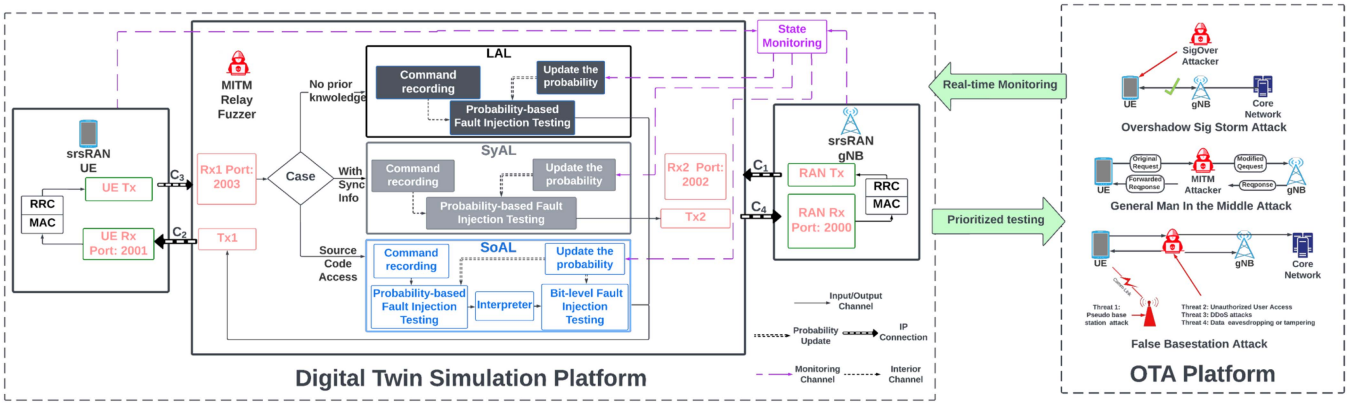


Fig. 1. Overall HyFuzz framework: Hybrid Digital Twin and OTA platform for monitoring and prioritizing high-risk areas in 5G communication. This figure serves as a roadmap for the system design discussed in later sections.

II. Background and Related Work

A. Fuzzing Models and Strategy

Fuzzing for communication protocols, especially wireless communication protocols, involves intercepting and modifying the data transmitted between two communicating parties to discover vulnerabilities such as data tampering, unauthorized access, or protocol weaknesses, similar to a man-in-the-middle (MITM) attack. A general fuzzing model can be represented as follows:

$$F(d, f) = f(d)$$

where:

- $F(d, f)$ is the fuzzed data or command d after being modified by the fuzzing function f.
- d is the original data or command.
- f represents the fuzzing function that generates the fuzzed output from d.

The fuzzing function f can operate in multiple modes:

a) Bit-level Fuzzing: Bit-level fuzzing involves modifying specific bits within the transmitted data to discover vulnerabilities such as buffer overflows, data corruption, or unexpected system behavior. The mathematical model can be represented as follows:

$$F_b(d, p) = d \oplus (1 \ll p)$$

where:

- $F_b(d, p)$ is the fuzzed data at position p.
- d is the original data.
- p is the position of the bit to be fuzzed.
- $\oplus$ represents the XOR operation.
- $1 \ll p$ shifts the bit 1 to the p-th position.

This model effectively flips the bit at position p in the data d.

b) Command-level Fuzzing: Command-level fuzzing manipulates the commands or messages exchanged within the network. By systematically altering the order, content, or parameters of these commands, we can expose vulnerabilities in the protocol logic, command parsing, and overall command processing flow.

The mathematical representation for command-level fuzzing can be formulated as:

$$F_c(C, s, p, r) = (C - c_s) \cup c_r$$

where:

- $F_c(C, s, p, r)$ is the fuzzed set of commands.
- C is the original set of commands.
- c_s is the command at sequence s to be replaced.
- p is the parameter of the command to be modified.
- c_r is the replacement command.

This model replaces the command c_s in the sequence with a new command c_r.

c) Permutation-Based Command-Level Fuzzing: Permutation-based command-level fuzzing involves rearranging the order of commands in the sequence to expose vulnerabilities in protocol logic, command parsing, and overall command processing flow. This method is useful when the attacker lacks the knowledge to generate new commands and instead rearranges existing commands to observe different behaviors.

The mathematical representation for permutation-based command-level fuzzing can be formulated as:

$$F_p(C, \pi) = C_\pi$$

where:

- $F_p(C, \pi)$ is the permutated set of commands.
- C is the original set of commands.
- π is a permutation of the indices of the commands.
- C_π is the sequence of commands after applying the permutation π.

B. Comparison and Related Work

Fuzzing approaches differ primarily by abstraction level and impact. **Bit-level** fuzzing flips individual bits to expose low-level faults (e.g., boundary violations, data corruption) and is computationally inexpensive but technically demanding and best performed in controlled testbeds to avoid damaging targets. [21], [22], [35] **Command-level** fuzzing mutates or forges protocol messages to probe control-plane logic and parsing; it is more

TABLE I
COMPARISON OF FUZZING TECHNIQUES

Fuzzing Technique	Attack Cost	Difficulty Level	Risk to Systems
Bit-Level Fuzzing	Low	High	Moderate
Command-Level Fuzzing	High	Moderate	High
Permutation-Based Command-Level Fuzzing	Moderate to High	Moderate to High	High

costly and less delicate than bit-level fuzzing but effective at finding logic and state-machine errors. [1], [25], [27] **Permutation-based** (sequence) fuzzing rearranges observed legitimate commands to reveal vulnerabilities arising from unexpected message ordering; its cost is similar to command-level methods, and its risk to system stability can be high when surprising sequences are applied. [17], [23], [28]

Table I summarizes these tradeoffs (attack cost, implementation difficulty, and operational risk) and motivates HyFuzz's hybrid, cross-layer approach that combines bit-level precision with session-aware command and permutation strategies.

III. Component Based Digital Twin Design for Extendable and Scalable HyFuzz System

The integration of digital twins into fuzz testing presents significant advancements in the field, particularly for complex and dynamic environments such as 5G and NextG networks. A digital twin, a virtual duplication of a physical device, system or process, allows for detailed simulation and analysis of real-world operations within a controlled, risk-free sandbox. By replicating the exact state and configuration of the physical network, digital twins enable comprehensive testing across various scenarios, including edge cases and rare conditions that are challenging to reproduce physically. Additionally, digital twins offer a cost-efficient solution by facilitating extensive testing in a virtual environment, thus eliminating the requirement for physical resources and reducing associated expenses. Testing within a digital twin also enhances safety by preventing potential attacks or unintended emergent behaviors that could occur in a real-world physical network.

Analogous to leveraging digital twins to improve system efficiency and reliability, component-based software architecture design is a modular approach where a system is decomposed into distinct components that can be developed, tested, and maintained independently. Each component encapsulates a specific functionality and communicates with other components through well-defined interfaces. This architecture promotes reusability, scalability, and maintainability, making it easier to manage complex systems.

Component-based digital twin design is crucial for the HyFuzz system. First, it enables integration with formal methods to optimize fuzzing seeds and strategies, enhancing vulnerability detection efficiency [12]. Second, it supports efficient data management for continuous monitoring and refining fuzzing strategies, improving accuracy [27]. Third, adaptability to real-world policies ensures relevant and applicable fuzzing results [25]. Finally, this design allows for creating close-to-real-world virtual environments for testing and proactive defense, even when details of system components are unavailable, as in [36] where they are proprietary. This approach builds upon our initial exploration in component-based digital twin design for a virtualized fuzzing system [37], and we have significantly extended it to enhance scalability and long-term effectiveness by enabling extensions to address emerging threats [27].

A. Guiding Fuzzing to High-Risk States Using Symbolic-Based Formal Methods

A key challenge in fuzz testing lies in determining where to focus testing efforts within a system's vast state space. Randomly starting from the initial system state is inefficient and often fails to uncover vulnerabilities hidden in more complex or evolved states. HyFuzz addresses this by leveraging symbolic-based formal methods to systematically identify high-risk states and guide fuzzing seed selection. These methods analyze the specifications of the system under test (SUT) to generate a formal representation of its operational behaviors [17], [38] and potential vulnerabilities. By modeling system transitions, dependencies, and constraints, HyFuzz can extract system behaviors and dependencies using symbolic representations, ensuring comprehensive coverage of critical operations. It can detect states associated with elevated security risks, such as those resulting from misconfigurations, unintended interactions, or protocol inconsistencies. This approach allows HyFuzz to focus fuzzing efforts on these high-risk states, ensuring that computational resources are allocated effectively.

B. Fuzzing Seed Selection for High-Risk States

Once high-risk states are identified, HyFuzz uses these insights to guide fuzzing seed selection, ensuring that fuzzing begins in states where vulnerabilities are more likely to exist. This significantly improves the efficiency and coverage of the testing process. Symbolic analysis identifies transitions leading to high-risk states, providing the basis for generating targeted fuzzing seeds. Continuous monitoring during fuzzing updates the formal model, refining seed selection and ensuring adaptability to new threats.

Consider the example of fuzz testing the authentication process in 5G protocols. Using symbolic-based formal methods, HyFuzz identifies high-risk states associated with the handling of authentication requests. These states include scenarios where the system transitions to a state where integrity checks are partially bypassed due to malformed authentication requests. Additionally, protocol parameters (e.g., K_{NASenc}, K_{NASint})

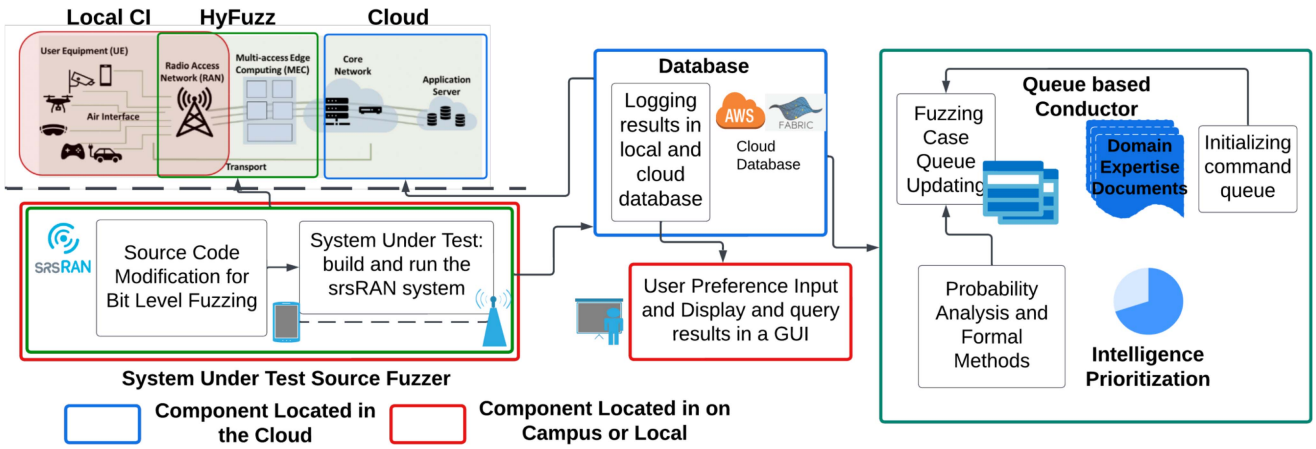


Fig. 2. The framework of Autonomous Fuzzing Digital Twin located at the campus of Stevens Institute of Technology, that connects local infrastructure and cloud computing.

may be improperly initialized, creating vulnerabilities for man-in-the-middle attacks. Symbolic-based formal methods detect these states by analyzing the logical flow of authentication procedures and identifying deviations from expected behaviors. Fuzzing seeds are then generated to simulate these high-risk conditions, allowing HyFuzz to uncover vulnerabilities more effectively than random seed generation.

C. Digital Twin's Architecture and Role in Testing and Operational Resilience

In addition to its role in fuzzing seed selection during the testing phase, the digital twin also plays a critical role in monitoring the operational system, as shown in Fig. 1. First, leveraging the probability-based Markov states learned during the testing phase, the communication system can prioritize and allocate more resources to high-risk states, thereby proactively mitigating potential attacks. By focusing computational and defensive efforts on these states, the system minimizes vulnerabilities and enhances overall security. Concurrently, the digital twin is capable of simulating subsequent system operations to predict the likelihood of risks before an attack occurs. This predictive capability enables preemptive interventions, reducing the probability of successful attacks. Second, in scenarios where the system is compromised, the digital twin provides a more efficient and systematic approach to risk mitigation compared to traditional Over-the-Air (OTA) systems. Unlike conventional methods, which often require costly and time-intensive processes to identify and replicate the attack scenario, the digital twin can swiftly simulate the compromised state and perform a comprehensive failure analysis. This includes pinpointing the root cause of the attack, identifying affected components, and generating actionable insights to address the vulnerability. Such capabilities not only expedite the recovery process but also improve the robustness of the system by feeding the insights back into the digital twin's learning process, ensuring continuous improvement and adaptation.

Fig. 2 presents an overview of the framework of the Autonomous Fuzzing Digital Twin located at the campus of Stevens

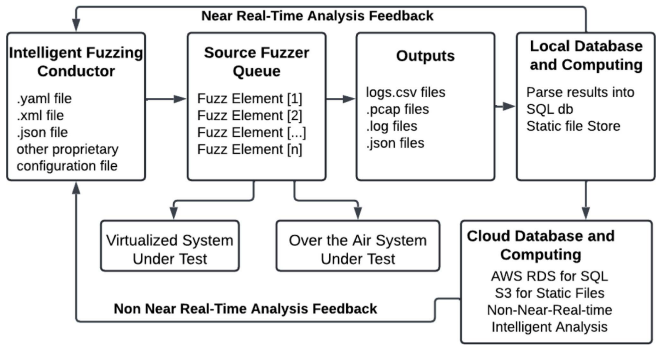


Fig. 3. The process from fuzz element to database.

Institute of Technology. This framework connects local infrastructure with cloud computing. Based on the assumptions identified by the hybrid formal methods, test cases are created to challenge these assumptions and are fed into the system for data analysis. This data is further utilized to build intelligent machine learning-based models capable of detecting patterns across various software stack implementations. The formal methods specifications developed in a flagship stack can be replicated across multiple implementations and stacks, ensuring consistency and reliability in the testing process. The scenario preference and user display, along with real-time query capabilities, are integrated into the graphical user interface (GUI) component. A detailed demonstration of the UI design and capabilities is presented in our prior demo paper [37]. The scenario preference includes considerations such as whether the operating scenario is within a private organization's local area network, a publicly accessible network, or an adversarial environment. These preferences impact the selection of the fuzzing strategy. Fig. 3 illustrates the process from the fuzz element to the database in the component-based digital twin design.

In OTA fuzzing, we assume an adversary capable of transmitting within the wireless range of the victim device, such as being co-located in the same cell coverage area or operating a rogue base station in proximity. We further assume the attacker

can synchronize with ongoing signaling procedures (e.g., attach, handover, or reconfiguration) to inject malformed messages at critical timing windows. While these assumptions are consistent with prior work in wireless security, they also reflect practical limitations: OTA fuzzing requires physical proximity, precise timing control, and the ability to emulate or impersonate legitimate network components. These factors bound the feasibility of attacks, but also underscore the real-world risks when adversaries possess modest technical resources.

The system integrates a customizable SUT source fuzzer within the srsRAN platform, enabling multi-level connections for real-time and zero-day vulnerability detection under the Delve-Yao model. Its database components support fast, extensible, and scalable data storage, leveraging AWS and Fabric for cloud integration. The local CI environment interfaces with HyFuzz, allowing bit-level fuzzing and testing through srsRAN, while cloud components, including MEC and core network elements, ensure seamless operation across diverse environments. A user-friendly GUI facilitates interaction, result querying, and test configuration. A queue-based conductor manages and prioritizes fuzzing cases using domain expertise, probabilistic analysis, and generative AI, while the fuzzing generator employs generative AI to optimize fuzzing strategies based on prior research and literature.

To tackle the challenges of efficient resource allocation and precise risk prediction in 5G fuzz testing, we developed a Markov chain-based approach that learns risk probabilities from digital twin simulations and continuously updates them during real-time OTA platform operations. The primary obstacles included integrating real-time data with simulated environments, managing computational resources effectively, and maintaining accuracy in risk prediction under uncertain conditions. By harnessing the Markov chain's capability to model state transitions, the system dynamically adapts to evolving risks, focuses resources on high-risk areas, and improves prediction accuracy.

D. Summary

The component-based digital twin design equips HyFuzz with a modular, extensible framework that mirrors real-world 5G environments. By integrating formal methods, symbolic analysis, and probabilistic modeling, it enables efficient seed selection, targeted fuzzing, and proactive monitoring of high-risk states. This foundation ensures that HyFuzz can adapt to emerging threats while maintaining scalability and realism.

IV. Development and Configuration of the Source Fuzzer: HyFuzz Platform

To enable the modification and injection of commands, a relay model is employed as the foundation for accessing the transmitted messages between the User Equipment (UE) and gNodeB (gNB). The HyFuzz system model hierarchically incorporates the advantages of two distinct platforms: the ZMQ virtual relay model and the OTA physical relay model. In the initial stage, HyFuzz utilizes the ZMQ virtual relay model to detect high-risk commands, leveraging its capabilities to provide an initial assessment of vulnerabilities. Subsequently, the system deepens the analysis by employing the OTA physical relay model, which extends the assessment of single-step vulnerabilities to multi-step vulnerabilities. By combining the efficiency of the ZMQ virtual relay model with the comprehensive coverage of the OTA physical relay model, HyFuzz offers a comprehensive and efficient framework for identifying and analyzing vulnerabilities in wireless communication systems. The following provides a summary of the HyFuzz framework, with a detailed description available in [39].

A. ZMQ Virtual Relay Model

The proposed virtual platform leverages the srsRAN framework to establish an IP relay using ZMQ over IP sockets, enabling robust and isolated environments for accurate vulnerability detection. Specifically, for core network, we employ srsRAN, which provides a 4G EPC core and a 5G NR gNB stack, corresponding to a Non-StandAlone (NSA) configuration. By abstracting hardware and supporting parallel computing, the platform ensures stability, efficient resource utilization, and agility in setup compared to OTA testing. It decodes raw IQ samples from the PHY layer of srsRAN into MAC PDUs using PyCrate and 3GPP protocols, allowing detailed analysis of transmitted messages.

A MITM relay node intercepts and redirects uplink signals between UE and gNB, addressing vulnerabilities in public wireless communications. This node performs data fetching, decoding, and interpretation while ensuring resilience against external noise. The platform integrates command-level and bit-level fuzzing techniques to evaluate system security comprehensively. These methods facilitate the identification of vulnerabilities and the assessment of robustness across various layers of abstraction.

B. OTA Cross-Layer Relay Model

As introduced earlier in Section III.C, OTA fuzzing assumes an adversary in range with capability to synchronize signaling procedures. Here, we extend this baseline by focusing on challenges specific to physical relay implementation. By establishing a realistic testing environment, the platform facilitates in-depth analysis of the vulnerabilities, their potential impacts, and the system's resilience against various attack scenarios. In the OTA mode, we configure srsRAN as the UE to control the USRP B210 device and facilitate communication with the AMARI Callbox, which acts as the gNB and core network (CN) components. The AMARI Callbox can be replaceable by srsRAN for an open programmable RAN and core network. However, considering the still exhibited instability and lack of performance testing benchmarks which creates challenges in converting a research result into a practical industry solution using current open source software defined radio platform, including srsRAN, in this paper, we use the industrial level 5G RAN acted by AMARI Callbox as benchmark framework. The communication log file is obtained from the AMARI Callbox via ssh into the AMARI Callbox through the switch to avoid any potential impacts on the under-test connection quality and performance. This setup enables seamless integration between the various components and allows efficient data exchange and

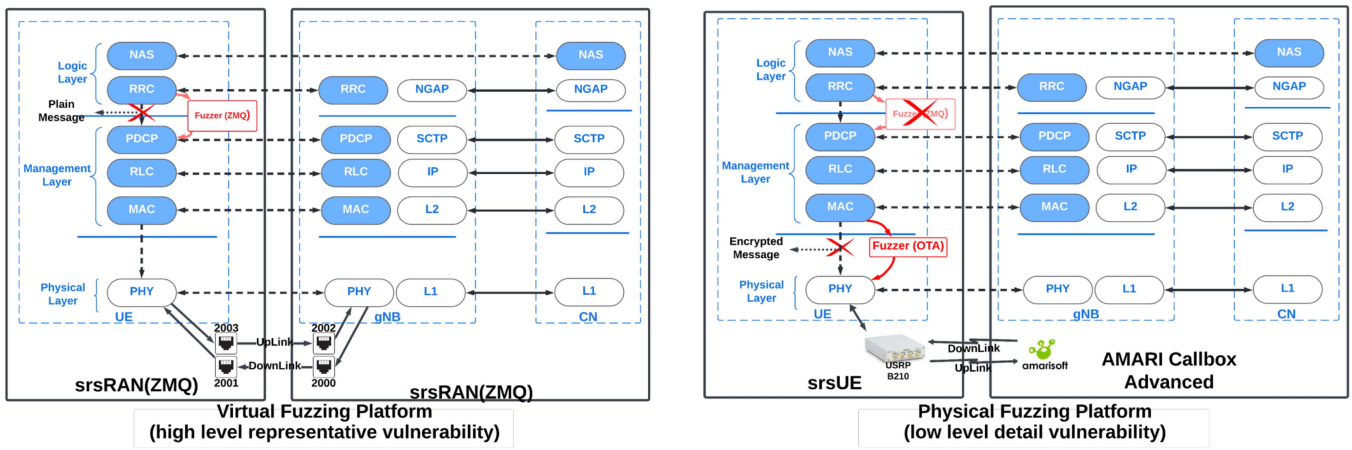


Fig. 4. Layer-based abstraction of HyFuzz platform.

analysis, ensuring a reliable and comprehensive experimental environment for conducting the desired investigations. To complement the OTA cross layer relay model, which captures end-to-end behaviors in real-world wireless settings, we next introduce higher-layer relay models that provide more abstract but still effective ways to simulate protocol vulnerabilities. These models bridge the gap between purely logical testing and full OTA physical testing, offering a layered perspective for fuzzing analysis.

1) Radio Resource Control (RRC) Logical Relay Model: In RRC-related vulnerability discovery, the state of the art 4G and 5G research, including LTEInspector [24], LTEFuzz [25], 5GReasoner [1], BaseSAFE [28], Berserker [15], T-Fuzz [23] and our previous work, Listen-and-Learn [27], implemented attacks only at the logical layer, more specifically at RRC layer, shown in Fig. 4. Compared to modifying the encrypted message transmitted between the UE and gNB, altering the unencrypted identifier in the RRC layer is an abstract yet efficient approach to vulnerability discovery in a virtual environment.

2) MAC Approximate to Physical Layer Relay Model: As described in previous section, fuzz testing is a more efficient approach to discover vulnerabilities. However, fuzzing in RRC layer presents limitations in simulating modification and injection attacks as any changes to identifiers also update the status of the UE. To overcome this limitation, we introduce a proximate to physical layer relay model in OTA mode that allows attack modification without altering the UE state. This enables the MAC proximate to the physical (PHY) relay model and allows for a more sophisticated vulnerability and unintended emergent behavior detection.

The MAC proximate to physical layer relay model offers a novel approach to simulate attack modification in 5G networks while preserving the integrity of the UE state. By operating at a proximate level to the PHY, this model enables the modification of command identifiers without triggering changes in the UE's status, like Fig. 4. This advancement overcomes the limitations of conventional attack simulation methods, which are typically confined to the logical layer and cannot accurately simulate attack modification and injection scenarios.

C. Achieving Depth in Fuzzing

Therefore, our proposed platform achieves "deep" in three critical directions, each addressing key aspects of a comprehensive and effective fuzzing framework:

1) *Multi-level Detection Approaches:* Our methodology adopts a multi-layered detection strategy that integrates command-level and bit-level fuzzing. Command-level fuzzing focuses on high-level inputs and interactions, enabling the detection of vulnerabilities that arise from logical flaws or protocol misconfigurations. On the other hand, bit-level fuzzing delves deeper into the granularity of individual bits and bytes, uncovering vulnerabilities that may result from low-level data handling issues, such as buffer overflows or memory corruption. This hierarchical approach ensures that our platform addresses a broad spectrum of vulnerabilities, from surface-level issues to deeply embedded flaws, providing a more thorough and holistic evaluation.
2) *Iterative Deep Exploration:* Unlike conventional single-pass detection methods, our platform employs a feedback-driven iterative mechanism for deep exploration. After each fuzzing iteration, the results are analyzed to generate insights and refine subsequent testing inputs. This feedback loop enables the platform to adapt its fuzzing strategy dynamically, targeting previously uncovered areas with greater precision. Over multiple iterations, the system achieves a more comprehensive and in-depth examination of the test subject, uncovering vulnerabilities that static or one-dimensional approaches might overlook. This purposeful and guided exploration enhances the detection rate and improves the fuzzing process's overall efficiency.

These three directions work synergistically to establish a platform that is not only "deep" in its capabilities but also innovative and robust in addressing the complexities of modern vulnerability detection challenges. Our approach sets a new standard for depth and effectiveness in fuzz testing by combining advanced simulation, granular multi-level analysis, and adaptive exploration.

D. Summary

Our dual relay design combines a ZMQ virtual relay with an OTA physical relay, enabling HyFuzz to balance efficiency with realism. The virtual relay supports rapid exploration and detection of high risk commands, while the OTA relay validates the virtual findings in a live wireless setting, revealing vulnerabilities that only appear in physical channels.

V. Implementation and Execution of the HyFuzz

In order to efficiently and comprehensively identify software unintended emergent defects and logical vulnerabilities, we employ two distinct fuzzing approaches: command-level fuzzing and bit-level fuzzing. These approaches enable the detection of vulnerabilities at different levels, providing a more thorough assessment. The key difference between command-level fuzzing and bit-level fuzzing lies in the granularity and focus of the fuzzing process. Command-level fuzzing explores the behavior of the system based on the commands or messages it receives, while bit-level fuzzing delves into the individual bits of the data stream. By employing both techniques, we can achieve a more comprehensive assessment of the software's vulnerability landscape, covering a wide range of potential weaknesses at different levels of abstraction. The subsequent sections illustrate the performance of various fuzzing strategies across different platforms, shedding light on their effectiveness and suitability for specific scenarios. The following summary provides an overview of the discussed concepts, with detailed explanations and methodologies available in [39].

A. Command-Level Fuzzing

1) Virtual Relay Mode: In the ZMQ virtual relay model, encrypted messages between UE and gNB pose challenges in decoding command types. To address this, an assumption-fixing command random replacement strategy is introduced, ensuring identical encryption keys under fixed assumptions. This strategy enables seamless command replacement and facilitates the analysis of system behavior.

2) OTA Physical Relay Mode: The OTA physical relay model extracts RRC layer information from the MAC PDU structure and interprets command headers. A probability-driven multi-step fuzzing strategy is implemented to detect multi-step vulnerabilities, targeting high-risk commands in the OTA environment.

B. Bit-Level Fuzzing

1) Virtual Relay Model: Bit-level fuzzing is applied in the RRC and MAC layers of the ZMQ virtual relay model. In the RRC layer, it modifies plaintext identifiers to track vulnerabilities effectively.

Alg. 1 illustrates the *command-level fuzzing logic* implemented by the virtual relay. The relay acts as an intelligent intermediary between the command receiver and transmitter during the connection process. It begins by checking for available fuzzing cases from the input database (I_1). For each case, it initiates a new communication session and resets the fuzzing flag.

Algorithm 1: ZMQ Virtual Relay Model Command-Level Fuzzing.

Input Database_query I_1, Command_receiver I_2
Output Database_update O_1, Command_transmitter O_2
procedure FUZZING_RELAY I_1
  while $exist\_fuzzing\_cases(I_1)$ **do**
    $start\_new\_communication()$
    $Fuzzed \leftarrow =$ False
    connection_history $h \leftarrow []$
    result $s \leftarrow$ False ▷ False means failed connection; True means successful connection
    while $!time\_out()$ **do**
      if $a = receive\_command(I_2)$ **then**
        if $type(a) = rrc\_Connection\_Complete$ **then**
          $s =$ True
          break
        if not $Fuzzed$ **then**
          $a = get\_candidate\_command(I_1)$
          $Fuzzed =$ True
        $send\_out(O_2, a)$
        $update\_log(h, a)$
    $update\_log(O_1, h, s)$

Algorithm 2: OTA Physical Relay Model Command-level Fuzzing.

Input Database_query I_1, Command_receiver I_2, Fuzzing_Strategy I_3
Output Database_update O_1, Command_transmitter O_2
procedure FUZZING_RELAY I_1
  while $exist\_fuzzing\_cases(I_1)$ **do**
    $start\_simulator()$
    connection_history $h \leftarrow []$
    fuzzing_strategy $f \leftarrow new\_fuzzing\_case(I_3)$
    while $!time\_out()$ **do**
      if $a = received\_command(I_2)$ **then**
        if $a_{original}, a_{replacement} = next\_fuzzing\_case(f)$ **then**
          if $a = a_{original}$ **then**
            $a = a_{replacement}$
            $pop(f)$
        $send\_out(O_2, a)$
        $update\_log(h, a)$
    $update\_log(O_1, h, s)$

The relay then monitors the connection process by observing incoming messages. A successful connection is identified by the reception of an `rrc_Connection_Complete` message. If a connection is established and fuzzing has not yet been applied, the relay selects a candidate command (a) from I_1, sends it to the transmitter (O_2), and logs the interaction. The result of the fuzzing attempt is then recorded in the output log (O_1).

This sequential, one-time fuzzing strategy ensures that each command is tested only once per session, allowing the

framework to systematically assess the resilience of the protocol stack under controlled and time-bounded conditions.

2) OTA Physical Relay Model: Based on the assessment of high risk in the RRC layer of the ZMQ virtual relay model, bit-level fuzzing in the OTA physical relay model will focus on high-risk commands and directly flip bits inside the protocol data unit (PDU), excluding the command header in the medium access control (MAC) layer.

The detailed procedure is outlined in Algorithm 2. In this setup, a physical relay operates alongside a simulator to inject fuzzed commands into the live communication stream. For each fuzzing case retrieved from the input database (I_1), the system initiates the simulator and determines the appropriate fuzzing strategy (I_3), which selects and prepares the next test case.

During execution, the relay monitors incoming commands. When a matching target command is received, it replaces the original payload with a fuzzed variant, according to the strategy defined by the pair $(a_{\text{original}}, a_{\text{replacement}})$. The fuzzed command is then sent to the command transmitter (O_2), and all actions and outcomes are logged in O_1.

Unlike bit-level fuzzing through overshadowing, bit-level fuzzing in the MAC layer can also simulate a physical attacker without synchronization and decryption of the wireless signal.

C. Summary

HyFuzz achieves depth through multi-level fuzzing (bit- and command-level), iterative exploration, and session-aware analysis. Its algorithms maintain protocol state history, enabling multi-step vulnerability discovery that traditional one-shot fuzzers miss. This design ensures a thorough and adaptive exploration of the protocol's attack surface.

VI. Experimental Assessment of Fuzz Testing

By conducting extensive experiments and simulations, we thoroughly evaluate the effectiveness and efficiency of two relay platforms: the ZMQ virtual relay platform and the OTA relay platform. Our evaluation encompasses the verification of detected vulnerabilities. Additionally, we assess the scalability and flexibility of these platforms to ensure their suitability across diverse network environments and deployment scenarios. The performance analysis of the platforms primarily focuses on two key applications: command-level fuzzing and bit-level fuzzing.

Our OTA platform is established using the AMARI Callbox Advanced as CN and the USRP B210 as the UE. The USRP B210 is controlled by our designed srsRAN-based fuzzing system running on the Ubuntu operating system. The average latency for authentication establishment is approximately 45000 ms. To ensure any delayed successful connections detected, we set the expiration limit to 60000 ms.

A. Comparison of Fuzz Testing in ZMQ and OTA Platform

During the experiment to validate the detected vulnerabilities in the ZMQ platform within the OTA platform, we discovered that many were unique to the OTA environment. This finding highlights the security challenges and potential risks of the OTA platform, underscoring the importance of tailored security measures and thorough testing in this context.

TABLE II
Command List

Command Index	Command Type
0	RRC connection request
1	RRC connection setup complete
2	UL information transfer
3	Security mode complete
4	UE capability information

TABLE III
Comparison of Fuzz Strategy in Different Platforms

Fuzz Strategy (Up-Link)	Virtual Fuzz Testing	Physical Fuzz Testing
command 0 → command 3	Failed connection	Repeat of command 0
command 1 → command 3	Failed connection	Failed connection

In practice, ZMQ-based fuzzing campaigns completed within several hours due to the software-only environment, while OTA campaigns required multiple days of execution because of hardware synchronization, timing constraints, and retries under noisy conditions. In our setup, latencies exceeding 40 seconds were considered vulnerabilities; this threshold can be configured differently in other scenarios, which may affect overall campaign duration.

1) Vulnerability Detected Via Command-Level Fuzzing: To provide an efficient overview assessment, we initially use the ZMQ virtual relay model to detect potential vulnerabilities. However, due to limited access to information within the commands of the ZMQ virtual relay model, we also apply communication time analysis and formal verification techniques. Then, the OTA physical relay model is applied to verify the detected high-risk commands. This comprehensive approach improves our ability to effectively assess and identify vulnerabilities within the 5G network environment.

In the ZMQ relay model, we ensure the consistency of commands by fixing critical identities, such as ISMI and RAND values. Conversely, in the OTA physical relay model, we directly extract the RRC hex message from the MAC layer without needing to consider encryption and decryption. Table II provides labeled examples of commands on which we perform command-level fuzzing across different platforms. As anticipated, the fuzzing cases conducted in the ZMQ relay model may yield different outcomes when applied to the OTA physical relay model. Table III illustrates this observation, highlighting that replacing command 1 with command 3 results in a failed connection in both virtual and physical fuzzing scenarios. However, the behavior of replacing command 0 with command 3 differs. Command 0 (*RRC Connection Request*) serves as the initial command for communication. In the ZMQ relay model, the successful arrival of command 0 is treated as an unavoidable occurrence. Conversely, in the OTA physical relay model, all

TABLE IV
COMPARISON OF DIFFERENT BIT-LEVEL STRATEGIES FOR RRC CONNECTION REQUEST

Identifier	Fuzzing Value	Fuzzing in RRC Layer	Fuzzing in MAC Layer
ue-Identity	00	successful	failed
	01	successful	failed
	10	successful	failed

ambiguous commands are repeated to counteract signal loss during transmission. This discrepancy in behavior between the two models showcases the importance of considering verification in the OTA physical relay model.

2) Vulnerability Detected via Bit-Level Fuzzing: The approach employed in bit-level fuzzing follows a similar methodology to that of command-level fuzzing. However, we discovered that modifying the RRC layer alone is insufficient to replicate a realistic relay attack due to the unintended modification of status information during the fuzzing process. This distinction is evident in Table IV, which presents examples of varying outcomes across different models. Fuzzing at the RRC layer does not impact the communication between the UE and the gNB since higher-level fuzzing directly affects the UE or gNB's status. In contrast, fuzzing closer to the physical layer only influences the current message's value, as lower layers primarily handle transport channels and packet packing. This approach closely emulates a genuine OTA relay attacker, capable of receiving, deciphering, modifying, and enciphering messages without notifying either communicating party. Furthermore, we have the capability to revert the modified values back to their normal state in the downlink (DL) channel. This comprehensive fuzzing strategy closely aligns with real-world attack scenarios, enabling a more accurate assessment of system vulnerabilities.

3) Detected Multi-Step Fuzzing Vulnerability: To demonstrate the advantages of HyFuzz's multi-step fuzzing capability, we present a representative case study in which multiple fuzzing actions are applied within a single connection session. This example illustrates how compounded mutations can reveal deeper protocol-level vulnerabilities that one-time fuzzing cannot. Unlike traditional fuzzing frameworks that perform isolated injections, HyFuzz maintains session context through a database-driven relay architecture, enabling it to apply targeted mutations over sequential protocol stages. Specifically, the platform tracks command history and connection state, allowing multiple fuzzing attempts to occur within a single attach or setup cycle.

As shown in Fig. 5, the fuzzing session begins with a normal `RRC Connection Request` from the UE to initiate the attach procedure. The MitM relay first forwards the legitimate `RRC Connection Setup Complete` message as part of the standard procedure, as illustrated in the figure. Immediately following this, HyFuzz performs the first mutation by injecting a forged `NAS Security Mode Complete` message. Because this NAS-layer message arrives earlier than expected, the gNB incorrectly assumes that the security procedure has already been completed and prematurely advances its protocol state. Under this false security context, the UE can continue by sending a legitimate `UL Information Transfer` message, which the gNB processes without completing the

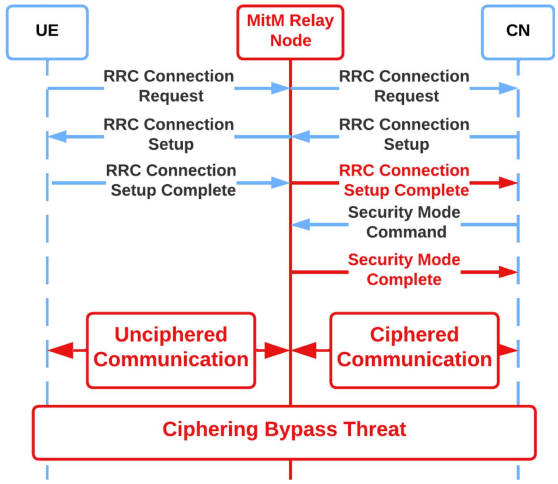


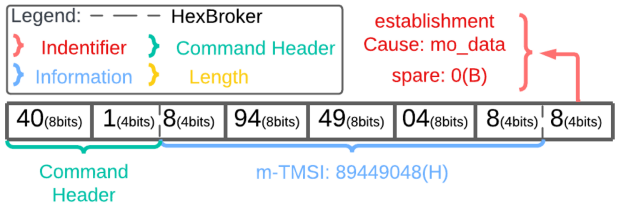
Fig. 5. Exposure via Multi-step Protocol State Desynchronization Attack. The MitM relay first forwards the legitimate RRC Connection Setup Complete as part of the normal procedure. Immediately following this, HyFuzz injects a forged NAS Security Mode Complete message (not shown explicitly in the figure). This additional NAS layer mutation causes the gNB to prematurely advance its protocol state, creating a transient desynchronization window in which subsequent protocol progressions may occur without proper cryptographic verification.

intended cryptographic handshake. In a second fuzzing action within the same session, HyFuzz injects another forged `NAS Security Mode Complete` message, which further reinforces the state desynchronization. The effect is that the gNB may temporarily accept protocol progressions without proper authentication. Although this inconsistency may eventually be detected by the UE through integrity check failures or security mode mismatch errors, by that time the gNB may already have processed unauthenticated control-plane messages. This demonstrates a transient state desynchronization vulnerability that can be exploited for control-plane manipulation or denial-of-service. Importantly, this behavior arises from protocol-level message ordering and session synchronization, not from a vendor-specific RAN implementation.

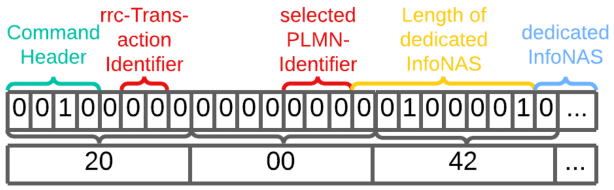
By observing the network behavior after these sequential injections, we detect an abnormal state where NAS messages are accepted by the gNB without proper authentication or encryption negotiation. This confirms a potential security state desynchronization vulnerability, where the gNB and UE operate under inconsistent assumptions about the security context. Notably, this vulnerability was not detectable through one-time or stateless fuzzing approaches, highlighting the critical role of HyFuzz's multi-step design in discovering stealthy, session-dependent flaws.

B. Bit-Level Fuzz Testing on OTA Platform

1) Fuzz Testing of RRC Connection Request: As shown in the example of the *RRC Connection Request* command in Fig. 6(a), the command can be roughly divided into four elements: command header, m-temporary mobile subscriber identity (TMSI), `establishmentCause`, and spare. To establish fuzz testing on this command, we set the command checker with the command header (`0x40 0x18`) to identify the *RRC Connection Request* command. If the command is our target command, we



(a) Content of *RRC Connection Request* command.



(b) Content of *RRC Connection Setup Complete* message.

Fig. 6. Example command/message structure. Solid lines are used to split hexadecimal numbers and dotted line are used to split octal numbers.

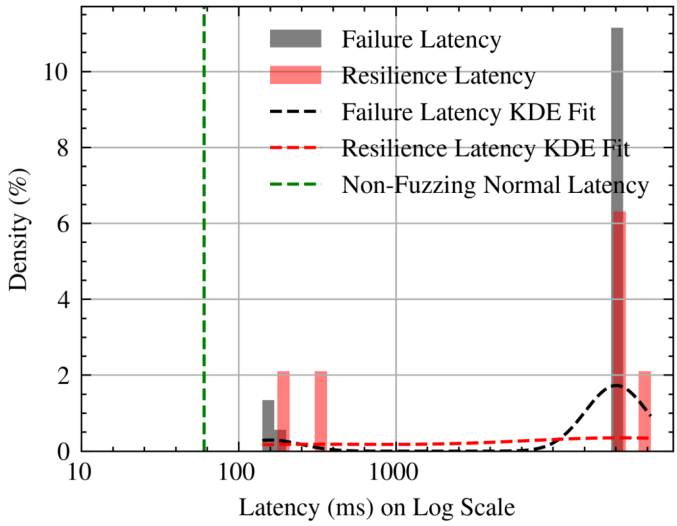


Fig. 7. Latency of fuzzing on RRC Connection Request. Two parts characterize the distributions of both resilience latency and failure latency. The shorter part is generated by rejecting duplicated requests. In comparison, the longer part is caused by connection release due to timer expiration.

will start fuzzing from the third hexadecimal number to the last hexadecimal number. All three potential fuzzing positions generate a total of 960 fuzzing cases. Except for three cases with the original number, most of the fuzzing cases show that the unverified TMSI and `establishmentCause` will lead to repeated *RRC Connection Request*s from the UE. However, there are still three fuzzing cases that change the command header from (`0x40 0x18`) to (`0x57 0x76`) to avoid the command checker. In these cases, the changed *RRC Connection Request* modifies `establishmentCause` to Mobile Originated Signaling (`mo-Signalling`). The subsequent attach request from the UE based on the changed command will be rejected by the CN. Therefore, we can conclude that overshadowing attacks on the *RRC Connection Request* command have a small probability of deriving a different *RRC Connection Request* command to establish an RRC connection, which is still rejected by the *Non-Access Stratum (NAS) Reject* command.

When analyzing the latency of fuzzing on *RRC Connection Request*, as illustrated in Fig. 7, we can identify two typical failure modes. The first type is a direct rejection by the CN, depicted by the green nodes at the bottom of Fig. 7. The second type involves repeated request attempts from the UE without any responses from the CN, continuing until the timer expires. The latency for all failure modes is significantly less than for successful cases because the authentication process is bypassed. Even within the successful cases, there are two subtypes: reverified mode, which has higher latency, and non-impact mode, which exhibits similar latency to non-attack scenarios. As expected, most failures lead to a repeat of *RRC Connection Request* and *RRC Connection Setup*. When counting the occurrence of commands, the frequency of these commands are significantly larger than others, as shown in Fig. 8, because most failures lead to these being repeated.

2) Fuzz Testing of RRC Connection Setup Complete*:* As the verification step of the RRC connection process, the *RRC Connection Setup Complete* command is responsible for the NAS

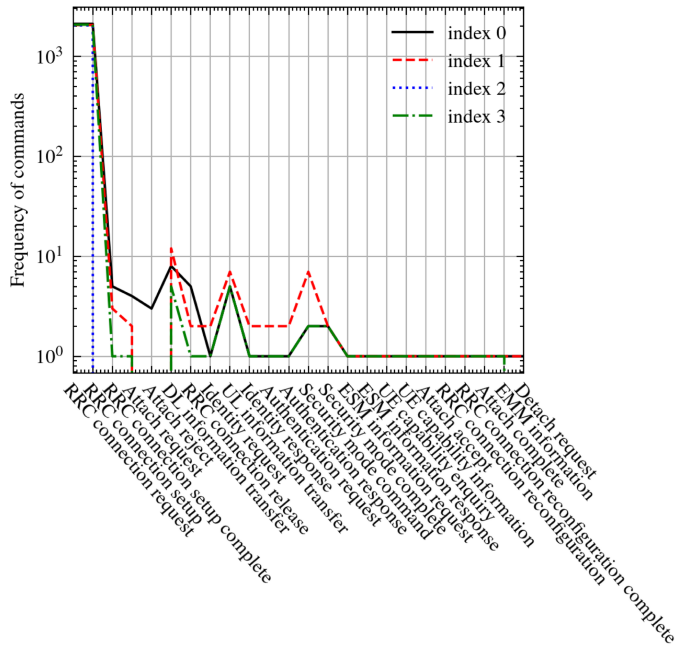


Fig. 8. The frequency of commands when fuzzing *RRC Connection Request*. The numerous *RRC Connection Request*s and *RRC Connection Complete*s indicate that almost every fuzzing attempt will generate multiple runs of *RRC Connection Request*s.

information update. As shown in Fig. 6(a), the first four header bits represent the identity of the command, and the following 11 bits represent the `rrc-TransactionIdentifier` and `selectedPLMNIdentifier`. The remaining part is used to store the NAS information. Therefore, the task of fuzzing *RRC Connection Setup Complete* can be split into two parts: fuzzing the identifier part (from the 5^{th} to the 15^{th} bit) and fuzzing the remaining InfoNAS (NAS information) part.

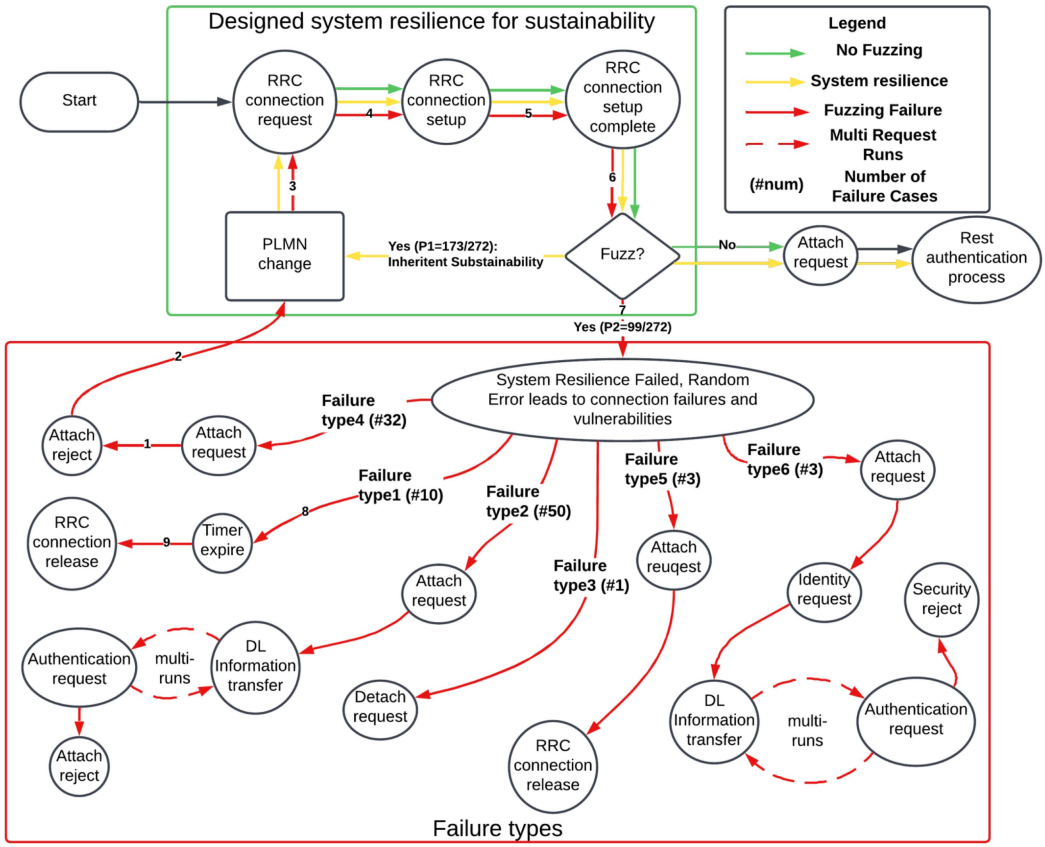
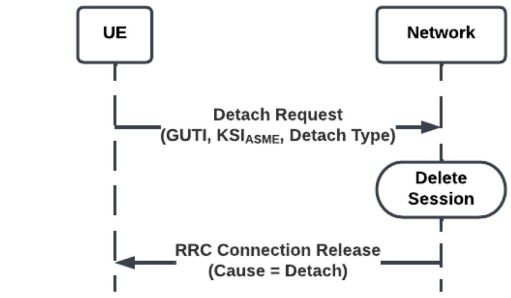


Fig. 9. Flow graph of the fuzzing result. In a typical process without fuzzing, only one instance of RRC authentication and NAS security setup occurs. System resilience helps the fuzzed request reconnect via additional connection requests. In contrast, the nodes in the lower part indicate various failed connection types.

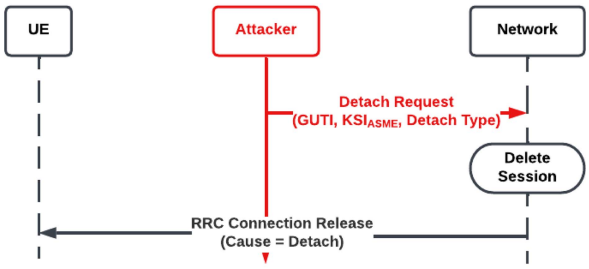
a) Fuzzing on identifier part of RRC Connection Setup Complete*:* In the identifier section, the legal range for `rrc-TransactionIdentifier` is 0 to 3, and for `selectedPLMNIdentifier`, it is 1 to 6. Since `selectedPLMNIdentifier` uses 3 bits, the CN will check if it exceeds the allowable range. Additionally, the *Length of dedicated InfoNAS* value must match the actual length of the InfoNAS section; otherwise, the integrity check will fail.

b) Fuzzing on InfoNAS part of RRC Connection Setup Complete*:* Through a total of 272 fuzzing cases, we encountered 99 failed connections. These 99 failures can be primarily categorized into 6 types:

1) *Direct RRC Connection Release from CN:* The process ends earlier than expected because of the NaN (Not Applicable/ No Action) step after *RRC Connection Setup Complete* from UE. This step indicates a point where no valid action is taken or where an unexpected or undefined behavior occurs. As a result, the connection process cannot proceed further and is prematurely terminated by the subsequent *RRC Connection Release* from the network DL, as shown in Fig. 9.
2) *Repeat Authentication Request and EPS Session Management (ESM) Information Request from CN:* After receiving the fuzzed *RRC Connection Setup Complete* message, the CN will send multiple authentication requests and ESM requests to verify the authentication of the UE, as shown in Fig. 9. Even if it receives an unfuzzed response from the UE, the CN will still reply with *Attach Reject* to decline the communication.
3) *Detach Request sent from UE:* The UE has security protocols that trigger *Detach Request* if the connection setup does not proceed as expected, mainly due to integrity check failures at the PHY layer in UE caused by fuzzed data.
4) *Attach Reject sent from CN:* The CN rejects the *Attach Request* from the UE due to a mismatch of fuzzed information. Then, the UE will restart connection requests which ultimately fail by way of a direct *RRC Connection Release* from CN. In Fig. 9, this process is shown by the arrows labeled 1-9.
5) *No reply to the Attach Request from the CN and the connection ends with timeout:* Instead of immediately rejecting the *Attach Request*, the CN may fail to interpret the information from the *Attach Request* and take no action to reply. After a timeout, the CN will send *RRC Connection Release* to end the connection.
6) *No reply to the Attach Request from the CN, but Identity Request is received, and the connection ultimately ends with Security Mode Reject from the UE:* This type of fuzzing will force the CN to send out multiple *Identity Request*s and *Authentication Request*s to verify the authentication of the UE, as shown in Fig. 9. However, when



(a) UE-initiated Detach procedure.



(b) Attack on UE-initiated Detach procedure.

Fig. 10. Attack model of detach request failure.

the CN trusts the UE and sends out the *Security Mode* command, the UE will respond with *Security Mode Reject* due to a mismatch in InfoNAS.

For example, we analyze the attack model of the *Detach Request*. In this attack, the adversary exploits a potential vulnerability in handling the Detach Request. By obtaining $GUTI$, KSI_{ASME}, and setting the Detach type as "Switch off," the network does not send a Detach Accept message to the UE, like shown in Fig. 10(a). Subsequently, the network initiates an $RRC\_Connection\_Release$ message to the UE.

Adversary Assumptions: The adversary possesses knowledge of $GUTI$ and KSI_{ASME} and is capable of setting up a Man-in-the-Middle (MitM) relay. This relay can eavesdrop on the victim's downlink messages from the UE and send a fake $detach\_request$ message.

Vulnerability: The root vulnerability of this attack lies in the lack of verification for the $detach\_request$ and $RRC\_Connection\_Release$ messages. This oversight enables the adversary to forge a $detach\_request$ message from the UE without raising any alarms.

Attack Description: The adversary uses a MitM relay to eavesdrop on downlink messages (e.g., $authentication\_request$) during the authentication process to extract the $GUTI$ and KSI_{ASME}. Leveraging this information, and with the capability of the relay, the adversary can initiate a detachment between the UE and the CN at any time, without being detected or questioned by the UE or CN, as show in Fig. 10(b).

By further analyzing the failures depicted in Fig. 11, we can conclude that failure type 2 has the largest quantity, followed by failure type 4. Other failures occur only a small percentage of the time. Regarding the distribution within the command structure, failure type 2 is evenly distributed among the InfoNAS part.

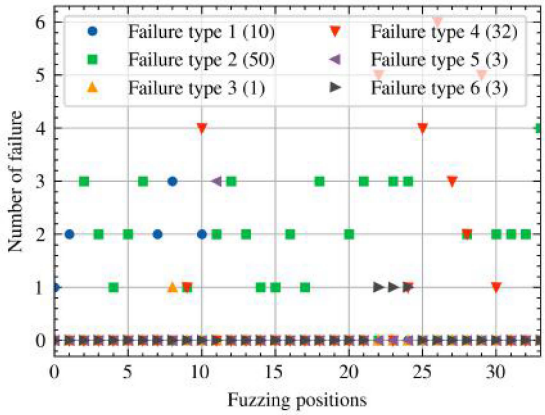


Fig. 11. Number of failures versus fuzzing position on *RRC Connection Setup Complete*. The total occurrences of each failure type across all fuzzing positions is shown parenthetically.

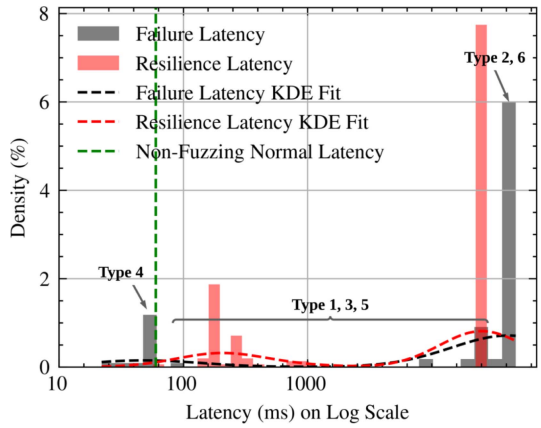


Fig. 12. Latency of fuzzing on *RRC Connection Setup Complete*. The distributions of resilience latency and failure latency also focus on two parts. The short failure latency is generated by direct rejection.

However, failure type 1 is concentrated at the beginning, while failure type 4 is concentrated at the end of InfoNAS.

As depicted in Fig. 12, the failure cases can be grouped into three major types:

1) The fuzzing attack is directly rejected by the CN (leftmost gray bar, which has the least latency and corresponds to failure type 4). This means that this type of fuzzing attack failure can be detected by system resilience.
2) The fuzzing attack is rejected after being ignored (gray bars in the middle, including failure types 1, 3, and 5). In this kind of system resilience failure, the system resilience ignores the fuzzed attack request, and the UE will release after a short period of waiting.
3) This type of fuzzing attack forces the UE to continue repeated authentication attempts until the authentication expires (rightmost gray bar, which has the longest latency and includes types 2 and 6). Most of the time, the authentication process becomes trapped in a loop of "Authentication request" and "DL Information transfer," which takes significantly more time than the other failure types.

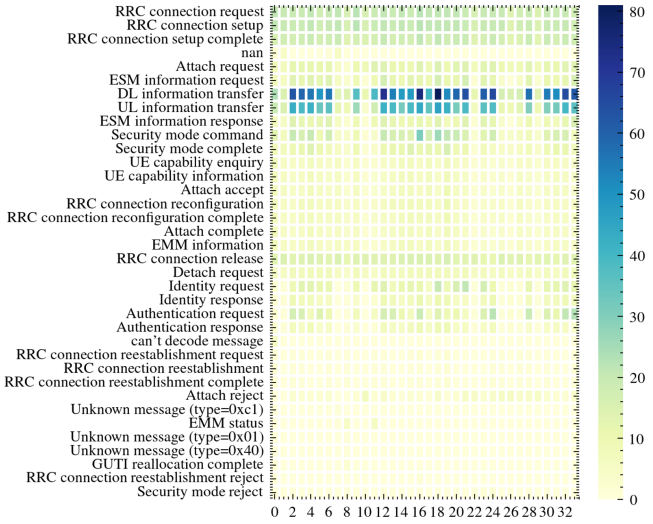


Fig. 13. Heatmap of frequency of message occurrence when fuzzing *RRC Connection Setup Complete*.

Most failures are due to repeated authentication attempts. Unlike failure cases, successful cases contain only two types: the first is the non-impact mode, which exhibits similar latency to non-attack scenarios, and the second, which constitutes the majority of cases, is the reauthentication mode. This mode requires twice the latency to establish authentication due to the need for reestablishment. In contrast to fuzzing on *RRC Connection Request*, the reestablishment of authentication based on fuzzing on *RRC Connection Setup Complete* triggers *DL Information Transfer* and *uplink (UL) Information Transfer*, which results in the frequency of these messages being significantly higher than others, as shown in Fig. 13.

C. Markov Chain-Based Failure Type Analysis

In the context of fuzz testing for NextG networks, understanding the behavior and transitions between different system states under attack scenarios is crucial. The unpredictability of multi-step attacks and the need to anticipate potential vulnerabilities necessitate a rigorous analytical approach. Markov chain analysis provides a systematic method to model the state transitions and predict the likelihood of system failures under fuzzing attacks.

The primary motivation for employing Markov chain analysis in this study is to gain a deeper understanding of the system's resilience when subjected to continuous and evolving attack strategies. By modeling the states and transitions, we can estimate the probability of the system entering a failure mode, identify critical states that lead to potential vulnerabilities, and predict the time it would take for the system to recover or fail. This analysis not only helps in identifying the weaknesses in the system but also assists in developing more robust defense mechanisms by anticipating attack patterns and improving the system's resilience.

Based on the counting numbers in Fig. 9, a Markov state diagram (Fig. 14) is generated to provide guidance on the probability of being attacked. We count the number of transitions from

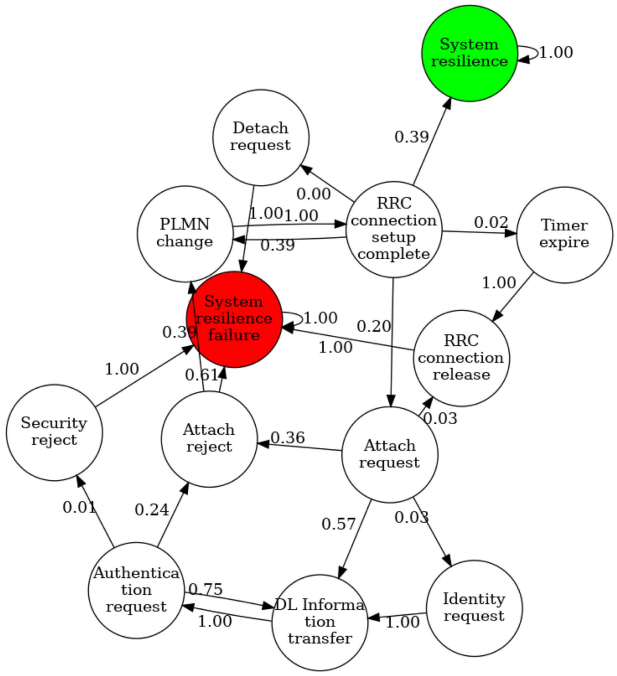


Fig. 14. Markov state of command state under fuzzing on *RRC Connection Setup Complete*.

each command i (state) to every other command j as T_{ij}. Then, the transition counts are converted to probabilities by dividing each count by the total number of transitions originating from the same state as follow equation:

$$P_{ij} = \frac{T_{ij}}{\sum_k T_{ik}}$$

This Markov state diagram represents the probabilities of state transitions when the user is unaware of whether an attack has occurred. Based on Markov state diagram, we can predict the latency through estimating the step before reaching an absorbing state, which, once entered, cannot be left. The expected number of steps from a transient state to an absorbing state can be represented as follows:

$$N = \sum_{k=0}^{\infty} Q^k = (I_t - Q)^{-1}$$

where Q is a t-by-t matrix that describes the probability of transitioning from one transient state to another, and I_t is the t-by-t identity matrix. All transition behaviors are treated as one unit of absorption time. When the probabilities of being in each state remain constant over time, after the long-term behavior of a Markov chain, the unchanged probabilities are the steady-state distribution. By analyzing the steady-state distribution, we can understand the likelihood of the system remaining resilient or failing in the face of persistent threats, allowing for better-informed decisions on enhancing security measures and mitigating risks.

The Markov state diagram identifies several critical states, notably *System Resilience* (green node) and *System Resilience Failure* (red node). *System Resilience* is an absorbing state,

TABLE V
EXPECTED TIME TO ABSORPTION FROM EACH TRANSIENT STATE

States	Expected time to absorption state
PLMN Change	0.6597
Attach Request	4.3379
Authentication Request	5.5082
DL Information Transfer	6.5082
Identity Request	7.5082

indicating that once the system reaches this state, it remains there, signifying robust system behavior. In contrast, *System Resilience Failure* is critical, with high transition probabilities from significant states like *Attach Reject*. The states connected to *System Resilience Failure* highlight vulnerable points where the system may falter.

Another significant observation is the loop between *DL Information Transfer* and *Authentication Request*, where each can transition to the other with a high probability. This loop can potentially delay the authentication process and failure notification, affecting the system's performance and resilience.

The steady-state distribution, which is generated after several transitions and finally remains unchanged in the Markov state diagram, focuses on the states *System Resilience Failure* and *System Resilience* with probabilities of 27.92% and 72.08%, respectively. This distribution suggests that while the system has a high likelihood of remaining resilient, there is still a significant risk (27.92%) of encountering a failure under persistent attacks.

To analyze the latency under a fuzzing attack on the command *RRC Connection Setup Complete*, we generate the expected time to absorption from each transient state, as shown in Table V. From Table V and Fig. 9, we can conclude that the most visited command, *Attach Request*, has a long expected time to absorption, and the multi-run loop components, including *Authentication Request* and *DL Information Request*, have longer expected times to absorption. However, without prior knowledge of the fuzzing attack path, the expected time of the state *Identity Request* is much longer than others because the only next transited states of state *Identity Request* is the loop of *Authentication Request* and *DL Information Request*.

Usually, the attack occurs in the UL channel and the UE has difficulty detecting it. For example, only failure type 3 involves a proactive process stop by the UE and it happens only once. However, by utilizing our presented Markov state diagram, the UE can predict the potential probability of an attack to avoid long latency before the authentication failure is detected. For example, when the UE enters the loop of *Authentication Request* and *DL Information Request* for the second time, it should stop and restart the authentication process to avoid wasting time.

D. Cross-References to Related Work

In addition to the results presented in this paper, the *HyFuzz* platform has been utilized in other works to detect vulnerabilities and report them to *3GPP through the Coordinated Vulnerability Disclosure (CVD) program*. For instance, the *Null Cipher Attack*, as described in prior work [40], was identified and validated using OTA validation, demonstrating how weak cipher negotiation protocols could expose sensitive user information. Similarly, the *Masquerade DoS Attack* (Section IV, Fig. 4 in related work) showcases how an adversary can manipulate RRC Setup Requests to force security context erasure, leading to denial-of-service conditions. Additionally, the *Incarceration Attack* (Section IV, Fig. 5(b)) highlights how message manipulation in OTA environments can create perpetual connection loops, resulting in resource exhaustion. These findings have directly informed recommendations to 3GPP to address gaps in protocol design, underscoring the broader impact of *HyFuzz* on real-world standards and applications.

E. Comparative Analysis With Existing Framework

Table VI offers a comprehensive comparison of the proposed HyFuzz platform with several prominent fuzzing methodologies, including 5GReasoner [26], LZFuzz [41], AMFuzz [42], Mutation-based Fuzz Testing [43], and MANDRAKE [44], among others. This comparison underscores the unique contributions of HyFuzz, particularly in its integration of advanced methodologies and its applicability to modern communication network verification challenges.

For consistency across tools, a detected vulnerability is defined as a unique fuzzing case that leads to unintended system behavior—such as persistent denial of service, inconsistent security state transitions, or unexpected protocol rejections—indicating a failure to gracefully handle edge-case or malformed inputs.

It is important to note that this comparison is qualitative rather than quantitative, since the cited fuzzers were originally deployed on diverse platforms and protocols, making a direct numerical benchmark under our 5G/NextG testbed unfair and inconsistent. In future work, we plan to adapt representative fuzzers to the same simulated and physical 5G environments used by HyFuzz, which will enable side-by-side quantitative benchmarking.

HyFuzz represents a significant advancement in network verification platforms, combining several features that distinguish it from existing approaches. A key strength of HyFuzz lies in its dual capability to support both simulation-based testing and OTA testing. This duality enables it to address a wide spectrum of scenarios, ranging from controlled virtual environments to real-world network evaluations, enhancing its versatility and practicality for emerging technologies such as 5G communication systems.

Another notable feature of HyFuzz is its implementation of multi-steps fuzzing, which systematically explores potential vulnerabilities through iterative processes. This approach allows HyFuzz to conduct a more exhaustive analysis than traditional fuzzing techniques. Furthermore, HyFuzz incorporates Markov probability models for risk prediction, providing a probabilistic framework to identify and evaluate vulnerabilities. This data-driven approach to risk modeling positions HyFuzz as a more precise and analytical tool than frameworks that do not incorporate such predictive mechanisms.

TABLE VI
COMPARATIVE ANALYSIS OF EXISTING METHODS FOR COMMUNICATION NETWORK VERIFICATION PLATFORMS

Model	Simulated Platform	Physical Platform	Risk Prediction	Multi-steps Fuzzing	Protocol	Formal Analysis	Prior knowledge	Machine Learning
5GReasoner [26]	✗	✗	Behavior-aware abstraction	✓	5G	✓	✓	✗
LZFuzz [41]	✓	✓	✗	✗	ICMP, HTTP	✗	✗	✗
AMFuzz [42]	✓	✗	✗	✗	5G	✗	✗	✗
Mutation-based Fuzz Testing [43]	✓	✓	✗	✓	TCP	✗	✓	✗
MANDRAKE [44]	✓	✓	Machine learning	✗	TCP, IoT	✗	✓	✓
PCPFuzz [45]	✓	✗	✗	✗	httpd, IoT	✗	✓	✗
SFO-CID [46]	✗	✓	Binary classification	✓	IoT	✗	✓	✗
FT-Framework [47]	✓	✗	N/A	✗	UAV	✗	✗	✗
HyFuzz	✓	✓	Markov probability/ Machine Learning	✓	5G	✓	Optional	✓

TABLE VII
DETECTED VULNERABILITIES THROUGH HYFUZZ PLATFORM

Number of detected vulnerabilities	Fuzzing platform	Total number of vulnerabilities
16 bit-level	Digital twin platform	1,105 in total
2 command-level		
19 bit-level	OTA platform	
2 command-level		
1,066 bit-level	formal guided fuzzing in OTA platform	

HyFuzz also distinguishes itself through its commitment to formal analysis, ensuring that vulnerabilities are detected and rigorously verified. This systematic approach lends credibility and robustness to its findings, a critical requirement for modern verification platforms. Additionally, HyFuzz's focus on the 5G protocol highlights its alignment with current technological trends and its ability to address the specific challenges posed by high-speed, low-latency networks. The platform's use of prior knowledge to guide its fuzzing processes further optimizes its efficiency, allowing it to target high-risk areas more effectively.

Based on the modest vulnerability counts reported by everal tools: 5GReasoner identified 20 issues, AMFuzz found 8, Mutation-based Fuzz Testing discovered 4, and SFO-CID reported 12 vulnerabilities in IoT contexts. In contrast, HyFuzz uncovered a significantly larger number of issues, identifying 1,105 vulnerabilities in total, including both command-level inconsistencies and over 1,000 b-level protocol anomalies through formal-guided OTA fuzzing, as shown in Table VII. These results highlight HyFuzz's ability to discover both structural logic flaws and fine-grained protocol deviations in complex multi-step interactions.

Despite its many strengths, HyFuzz has some limitations, the most prominent of which is its current focus on the 5G protocol. Although this specialization enables HyFuzz to achieve exceptional depth and optimization within this domain, it limits the applicability of the platform to other protocols, such as ICMP, HTTP, or IoT, which are supported by some of its counterparts. Expanding its protocol support could enhance its versatility. However, this trade-off underscores HyFuzz's design philosophy of excelling within a defined and critical domain.

F. Summary

HyFuzz is a robust and advanced framework for communication network verification, combining innovative features such as simulation and OTA support, multi-steps fuzzing, risk prediction, formal analysis, and prior knowledge utilization. Although its specialization in 5G protocol verification may limit its scope, this focus enables it to deliver unparalleled depth and reliability in addressing the complexities of 5G networks, making it a highly effective solution for contemporary verification challenges.

VII. CONCLUSION

This paper introduces *HyFuzz*, a robust and innovative hybrid testing platform that addresses critical limitations in existing fuzz testing frameworks for 5G and NextG networks. By integrating digital twins, multi-step fuzzing, and formal-guided methodologies, HyFuzz achieves a transformative capability to emulate near-real-world attack scenarios and systematically uncover vulnerabilities in both virtualized and over-the-air (OTA) environments. The experimental results demonstrate the platform's exceptional ability to identify 1,105 failure cases among 1281 test cases, encompassing 16 distinct types of vulnerabilities, with significant contributions to the detection of multi-step and OTA-specific attack strategies that remain challenging for traditional methods. The use of Markov chain-based state modeling enables HyFuzz to predict high-risk transitions, proactively mitigating vulnerabilities before they manifest in live deployments. Additionally, the digital twin framework supports the creation of realistic virtual replicas of physical systems, facilitating systematic risk analysis and actionable insights into system resilience under dynamic and contested conditions.

The findings from this study establish *HyFuzz* as a foundational tool for advancing cybersecurity in ultra-reliable and low-latency communication systems, bridging the gap between

theoretical validation and real-world applicability. Looking ahead, we will enhance *HyFuzz*'s capabilities by integrating machine intelligence-driven advancements to deepen its applicability to emerging network architectures. Specifically, we propose incorporating *Large Language Models (LLMs)* to automate the fuzzing seed selection process. Unlike conventional methods, LLMs offer the potential to analyze large-scale protocol documentation, extract high-risk areas, and adaptively generate tailored fuzzing seeds, thus enhancing testing precision and efficiency. Furthermore, we will employ *Monte Carlo Tree Search (MCTS)* to model complex multi-step interactions, enabling *HyFuzz* to simulate and detect vulnerabilities associated with evolving 6G network paradigms, such as integrated sensing and communication (ISAC) and terahertz communications. These directions will not only enhance *HyFuzz*'s automation and scalability but will also position it as a critical enabler of future research into adversarial modeling for heterogeneous and highly dynamic networks.

The next phase of development will also explore extending *HyFuzz* beyond 5G protocols to cover cross-domain vulnerabilities in 6G-enabled systems, including autonomous Internet of Things (IoT) ecosystems, vehicular networks, and satellite-terrestrial integration. This broader focus will address challenges in multi-protocol interactions, such as hybrid edge-cloud orchestration and federated learning vulnerabilities, which are poised to emerge as critical concerns in next-generation communications. By aligning these enhancements with emerging use cases, *HyFuzz* will continue to play a pivotal role in ensuring the security and reliability of future networked systems.

ACKNOWLEDGMENT

The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of DARPA, United States Air Force (USAF) or the US Government.

DISTRIBUTION STATEMENT

A. Approved for public release; distribution unlimited. AFRL-2024-4704, 26 Aug 2024.

REFERENCES

[1] Z. Salazar, H. N. Nguyen, W. Mallouli, A. R. Cavalli, and E. M. Montes De Oca, "5Greplay: A 5G network traffic fuzzer - application to attack injection," in *Proc. ACM Int. Conf. Proc. Ser.*, 2021, pp. 1–8.

[2] D. Basu, R. Datta, and U. Ghosh, "Softwarized network function virtualization for 5G: Challenges and opportunities," in *Internet of Things and Secure Smart Environments: Successes and Pitfalls*, vol. 147. Boca Raton, FL, USA: CRC Press, 2020.

[3] X. Foukas, M. K. Marina, F. Sardis, M. A. Lema, F. Foster, and M. Dohler, "Experience building a prototype 5G testbed," in *Proc. Workshop Experimentation Meas. 5G*, 2018, pp. 13–18.

[4] S. Yuan and C. Lipizzi, "AI-augmented literature reviews: Efficient clustering and summarization for researchers," *IEEE Access*, vol. 13, pp. 156535–156565, 2025.

[5] F. Kaltenberger et al., "The OpenAirInterface 5G new radio implementation: Current status and roadmap," in *Proc. WSA 2019; 23rd Int. ITG Workshop Smart Antennas*, VDE, 2019.

[6] I. Gomez-Miguelez, A. Garcia-Saavedra, P. D. Sutton, P. Serrano, C. Cano, and D. J. Leith, "srsLTE: An open-source platform for LTE evolution and experimentation," 2016, *arXiv:1602.04629*. [Online]. Available: http://arxiv.org/abs/1602.04629

[7] P. Valente et al., "Easy to deploy UAV-based non-public open-source 5G network," in *Proc. 15th Int. Conf. Netw. Future*, 2024, pp. 72–80.

[8] A. Moglia et al., "5G in healthcare: From COVID-19 to future challenges," *IEEE J. Biomed. Health Informat.*, vol. 26, no. 8, pp. 4187–4196, Aug. 2022.

[9] D. Li, "5G and intelligence medicine—how the next generation of wireless technology will reconstruct healthcare?," *Precis. Clin. Med.*, vol. 2, no. 4, pp. 205–208, 2019.

[10] A. Ahad, M. Tahir, M. Aman Sheikh, K. I. Ahmed, A. Mughees, and A. Numani, "Technologies trend towards 5G network for smart health-care using IoT: A review," *Sensors*, vol. 20, no. 14, 2020, Art. no. 4047.

[11] GSMA, "Network equipment security assurance scheme (NESAS) - overview (Version 3.0)," GSMA, Jan. 2022. [Online]. Available: https://www.gsma.com/security/networkequipment-security-assurance-scheme

[12] E. K. K. Edris, M. Aiash, and J. K.-K. Loo, "Formal verification and analysis of primary authentication based on 5G-AKA protocol," in *Proc. 7th Int. Conf. Softw. Defined Syst.*, 2020, pp. 256–261.

[13] H. Moudoud, L. Khoukhi, and S. Cherkaoui, "Prediction and detection of FDIA and DDoS attacks in 5G enabled IoT," *IEEE Netw.*, vol. 35, no. 2, pp. 194–201, Mar./Apr. 2021.

[14] H. Hu, J. Liu, Y. Zhang, Y. Liu, X. Xu, and J. Tan, "Attack scenario reconstruction approach using attack graph and alert data mining," *J. Inf. Secur. Appl.*, vol. 54, 2020, Art. no. 102522.

[15] S. Potnuru and P. K. Nakarmi, "Berserker: ASN. 1-based fuzzing of radio resource control protocol for 4G and 5G," in *Proc. 17th Int. Conf. Wireless Mobile Comput., Netw. Commun.*, 2021, pp. 295–300.

[16] J. Yang, Y. Wang, Y. Pan, and T. X. Tran, "Systematic meets unintended: Prior knowledge adaptive 5G vulnerability detection via multi-fuzzing," 2023, *arXiv:2305.08039*.

[17] J. Yang, S. Arya, and Y. Wang, "Formal-guided fuzz testing: Targeting security assurance from specification to implementation for 5G and beyond," *IEEE Access*, vol. 12, pp. 29175–29193, 2024.

[18] A. Turner, "Tcpreplay," 2011. [Online]. Available: http://tcpreplay.synfin.net/trac/

[19] M. Sústrik, P. Hintjens, and M. Kogut, "ZeroMQ," in *The Architecture of Open Source Applications, Volume II: Structure, Scale, and a Few More Fearless Hacks*, A. Brown and G. Wilson Eds., 2015, pp. 15–28.

[20] Q. Li, S. Meng, S. Wang, J. Zhang, and J. Hou, "CAD: Command-level anomaly detection for vehicle-road collaborative charging network," *IEEE Access*, vol. 7, pp. 34910–34924, 2019.

[21] H. Wang et al., "An automated vulnerability detection method for the 5G RRC protocol based on fuzzing," in *Proc. 4th Int. Conf. Adv. Comput. Technol.*, 2022, pp. 1–7.

[22] L. J. Moukahal, M. Zulkernine, and M. Soukup, "Vulnerability-oriented fuzz testing for connected autonomous vehicle systems," *IEEE Trans. Rel.*, vol. 70, no. 4, pp. 1422–1437, Dec. 2021.

[23] W. Johansson et al., "T-Fuzz: Model-based fuzzing for robustness testing of telecommunication protocols," in *Proc. IEEE 7th Int. Conf. Softw. Testing, Verification Validation*, 2014, pp. 323–332.

[24] S. Hussain, O. Chowdhury, S. Mehnaz, and E. Bertino, "LTEInspector: A systematic approach for adversarial testing of 4G LTE," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018.

[25] H. Kim, J. Lee, E. Lee, and Y. Kim, "Touching the untouchables: Dynamic security analysis of the LTE control plane," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 1153–1168.

[26] S. R. Hussain, M. Echeverria, I. Karim, O. Chowdhury, and E. Bertino, "5Greasoner: A property-directed security and privacy analysis framework for 5G cellular network protocol," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2019, pp. 669–684.

[27] J. Yang, Y. Wang, T. X. Tran, and Y. Pan, "5G RRC protocol and stack vulnerabilities detection via listen-and-learn," in *Proc. IEEE 20th Consum. Commun. Netw. Conf.*, 2023, pp. 236–241.

[28] D. Maier, L. Seidel, and S. Park, "BaseSAFE: Baseband sanitized fuzzing through emulation," in *Proc. 13th ACM Conf. Secur. Privacy Wireless Mobile Netw.*, 2020, pp. 122–132.

[29] A. Fuller, Z. Fan, C. Day, and C. Barlow, "Digital Twin: Enabling technologies, challenges and open research," *IEEE Access*, vol. 8, pp. 108952–108971, 2020.

[30] E. H. Glaessgen and D. S. Stargel, "The digital twin paradigm for future NASA and US air force vehicles," in *Proc. 53rd Structures, Struct. Dyn. Mater. Conf.: Special Session Digit. Twin*, 2012, Art. no. 1818.

[31] Department of Defense Digital Engineering Strategy. Office of the Deputy Assistant Secretary of Defense for Systems Engineering, Jun. 2018. [Online]. Available: https://ac.cto.mil/wp-content/uploads/2019/06/2018-Digital-Engineering-Strategy_Approved_PrintVersion.pdf

[32] DoD Instruction 5000.97, Digital Engineering. U. S. Department of Defense, Dec. 2023. [Online]. Available: https://www.esd.whs.mil/Portals/54/Documents/DD/issuances/dodi/500097p.PDF
[33] Q. Liu et al., "Digital twin-driven manufacturing cyber-physical systems: A review," *IEEE Access*, vol. 7, pp. 119601–119614, 2019.
[34] M. Grieves and J. Vickers, "Digital Twin: Mitigating unpredictable, undesirable emergent behavior in complex systems," in *Transdisciplinary Perspectives on Complex Systems*. Berlin, Germany: Springer, 2017, pp. 85–113.
[35] S. Potnuru and P. K. Nakarmi, "Berserker: ASN.1-based fuzzing of radio resource control protocol for 4G and 5G," in *Proc. 17th Int. Conf. Wireless Mobile Comput.*, 2021, pp. 295–300.
[36] A. D'Angelo, C. Fu, X. Du, and P. Ratazzi, "Discovering complex correlations among multiple IoT devices in smart environments," in *Proc. IEEE Glob. Commun. Conf.*, 2023, pp. 1914–1919.
[37] D. Dauphinais et al., "Automated vulnerability testing and detection digital twin framework for 5G systems," in *Proc. IEEE 9th Int. Conf. Netw. Softwarization*, 2023, pp. 308–310.
[38] J. Yang and Y. Wang, "Trustworthy formal verification in 5G protocols: Evaluating generative and classification models for relation extraction," in *Proc. IEEE Mil. Commun. Conf.*, 2024, pp. 956–962.
[39] J. Yang and Y. Wang, "HyFuzz: A NextG hybrid testing platform for multi-step deep fuzzing and performance assessment from virtualization to over-the-air," in *Proc. IEEE 12th Int. Conf. Cloud Netw.*, 2023, pp. 274–280.
[40] T. Wray and Y. Wang, "5G specifications formal verification with over-the-air validation: Prompting is all you need," in *Proc. IEEE Mil. Commun. Conf.*, 2024, pp. 412–418.
[41] S. Bratus, A. Hansen, and A. Shubina, "LZfuzz: A fast compression-based fuzzer for poorly documented protocols," in *Proc. 2nd USENIX Workshop Offensive Technol.*, Jul. 2008.
[42] F. Mancini, S. Da Canal, and G. Bianchi, "Amfuzz: Black-box fuzzing of 5G core networks," in *Proc. 19th Wireless On-Demand Netw. Syst. Serv. Conf.*, 2024, pp. 17–24.
[43] X. Han, Q. Wen, and Z. Zhang, "A mutation-based fuzz testing approach for network protocol vulnerability detection," in *Proc. 2012 2nd Int. Conf. Comput. Sci. Netw. Technol.*, 2012, pp. 1018–1022.
[44] U. Brezolin, A. Vergütz, and M. Nogueira, "A method for vulnerability detection by IoT network traffic analytics," *Ad Hoc Netw.*, vol. 149, 2023, Art. no. 103247.
[45] L. Li, R. Xu, L. Chen, B. Dong, W. Chen, and X. Xu, "PCPFuzz: Path coverage protocol based firmware fuzzing system," in *Proc. 5th Int. Acad. Exchange Conf. Sci. Technol. Innov.*, 2023, pp. 76–82.
[46] X. Chen, L. Sha, J. Wang, F. Xiao, and J. Dong, "SFO-CID: Structural feature optimization based command injection vulnerability discovery for Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 21, no. 2, pp. 1429–1438, Feb. 2025.
[47] J. Jang et al., "Fuzzability testing framework for incomplete firmware binary," *IEEE Access*, vol. 11, pp. 77608–77619, 2023.

Jingda Yang (Graduate Student Member, IEEE) received the BE degree in software engineering from Shandong University and the MSc degree in computer science from The George Washington University. He is currently working toward the PhD degree with the School of System and Enterprises, Stevens Institute of Technology. His research interests are formal verification and vulnerability detection of wireless protocol in 5G.

Paul Ratazzi (Senior Member, IEEE) received the BS degree in electrical engineering and the MS degree in management from Rensselaer Polytechnic Institute, and the MS and PhD degrees in electrical and computer engineering from Syracuse University. He is currently a principal engineer with the Information Warfare Division of Air Force Research Laboratory, and an adjunct with the Dept. of Network and Computer Security, SUNY Polytechnic Institute. His current research and teaching interests include securing edge devices, hardware-enforced security, secure protocols, and penetration testing. He is a past chair of IEEE's Mohawk Valley Section in Region 1.

Ying Wang (Member, IEEE) received the BE degree in information engineering from the Beijing University of Posts and Telecommunications, the MS degree in electrical engineering from the University of Cincinnati and the PhD degree in electrical engineering from Virginia Polytechnic Institute and State University. She is an associate professor with the School of System and Enterprises, Stevens Institute of Technology. Her research areas include cybersecurity, wireless AI, edge computing, health informatics, and software engineering.